Intro to R, Demonstration

A brief introduction to R packages

Packages are curated collections of functions and/or compiled code for use in R. Some commonly-downloaded, popular packages include sample data and documentation to explain and demonstrate implementation.
Packages can be downloaded from the Comprehensive R Archive Network (CRAN), the primary repository for R packages. Each package submitted to CRAN is reviewed and tested thoroughly before release. Every package is released with a version number, and the most popular packages are frequently updated and released with a new version number.
CRAN is a trusted source for up-to-date, consistent versions of R packages, and allows package accessibility across multiple servers in multiple regions called 'mirrors'. Packages can also be downloaded through GitHub, but we'll get to that later.

Installing a package is like buying a textbook (once), and `library()` is taking it off the shelf (every session).

Shortcut for inserting a chunk: `Ctrl+Alt+I` (Mac: `Cmd+Option+I`). Shortcut for running the current line or selection: `Ctrl+Enter` (Mac: `Cmd+Return`).
Version numbers matter for reproducibility. A methods section should report R and package versions, which you can get from `sessionInfo()`.
View the documentation for `sessionInfo()` by clicking **here.**

Two free resources worth bookmarking:
The online book *R for Data Science* by clicking **here.**
The RStudio/Posit cheat sheets (one-page summaries of common functions) by clicking **here.**

A note on the # symbol inside code chunks: any text after a # on a line is a **comment**. R ignores it. Comments are notes for humans reading the code.

1. Using `install.packages()`
   One of the most useful packages in R is `tidyverse`.
   Install `tidyverse` by typing `install.packages('tidyverse')` in the code chunk below.
   The `install.packages()` function is the easiest method to download packages from CRAN.
   The line below starts with #, so it is "commented out" and will not run. That is on purpose: you only need to install a package once per computer, and re-installing every time you knit the document is slow. To install, delete the # and run the line once, then put the # back.
   `tidyverse` is actually a bundle of several packages (`dplyr`, `ggplot2`, and others) that share a common style. Installing it can take a few minutes.
   View the documentation for `install.packages()` by clicking **here.**
   Read more about the tidyverse by clicking **here.**

```
# install.packages('tidyverse')
```

2. The `library()` function
   Once the package is downloaded, the package must be called into virtual memory before it is used.

In order to use the package, call it into virtual memory with the `library()` command.
When downloading packages, the package name must be in quotation marks. When called into virtual memory with the `library()` command, no quotation marks are needed.

Put the `tidyverse` package into virtual memory by typing `library(tidyverse)` in the code chunk below.
The text inside the curly braces at the top of the chunk, `message=FALSE, warning=FALSE`, is a set of **chunk options**. They tell R Markdown not to print messages and warnings from this chunk in the knitted document. The code still runs normally.
View the documentation for `library()` by clicking **here.**
Read more about chunk options by clicking **here.**

```
library(tidyverse)
```

You can also run `library(tidyverse)` in the console. "Conflicts" lines are normal and are not errors. Red text in R is not always an error.
A conflict means two loaded packages have a function with the same name (for example, both `dplyr` and base R's `stats` package have a `filter()` function). R uses the most recently loaded one.

3. Viewing package version with `packageVersion()`
   `packageVersion()` takes a package name in quotation marks and returns the version number that is installed on your computer, for example `2.0.0`.
   Your version may be different from your neighbor's if you installed at different times. This is why reporting versions matters.
   View the documentation for `packageVersion()` by clicking **here.**

```
packageVersion('tidyverse')
```

```
## [1] '2.0.0'
```

3a. Viewing the working directory with `getwd()`
You can view the working directory with `getwd()`. The working directory is a default file path for any data you would like to save, for example, a dataframe saved as a .csv. It is also the first place R looks when you load a file by name only.
`getwd()` takes no arguments, so the parentheses stay empty. The output is a file path in quotation marks.
View the documentation for `getwd()` by clicking **here.**

```
getwd()
```

```
## [1] "C:/Users/steph/OneDrive/CUNY_CSOM/SOHLER"
```

3b. Setting the working directory with `setwd()`
Set the working directory with the `setwd()` command. Remember to use forward slashes (`/`) or doubled back slashes (`\\`) in file paths. A single back slash means something special to R, so `"C:\Users"` will cause an error.
Setting the working directory saves the file path to the folder with all of your relevant files. Alternately, include file paths with file names when loading or saving files.

Change the path below to a folder on your own computer, or R will return an error ("cannot change working directory").
An important R Markdown detail: when you knit, `setwd()` only applies to the chunk it is in. The next chunk goes back to the folder the .Rmd file is saved in. This is one reason the next step uses the full file path. A tidier long-term habit is to keep your .Rmd file and data in the same folder, or to use an RStudio Project.
View the documentation for `setwd()` by clicking **here.**
Read about RStudio Projects by clicking **here.**

```
setwd("C:/Users/steph/OneDrive/CUNY_CSOM")
```

4. Importing a .csv file with `read.csv()`
   A .csv ("comma-separated values") file is a plain-text spreadsheet: each line is a row, and commas separate the columns.
   `read.csv()` reads the file and turns it into a **dataframe**, R's version of a spreadsheet, with rows and columns.
   The file path is in quotation marks and uses doubled back slashes (see Step 3b). Change it to the location of the file on your computer.
   View the documentation for `read.csv()` by clicking **here.**

```
brtri <- read.csv("C:\\Users\\steph\\OneDrive\\CUNY_CSOM\\SOHLER\\brfss2024tri.csv")
```

`<-` means "store the result in a box named `brtri`." It will appear in the Environment pane with its row and column counts.
Misspelling the file name in the Console: error ("cannot open file… No such file or directory"); pay attention to the working directory with `getwd()`.
Click the table icon next to `brtri` in the Environment pane (or run `View(brtri)`) to show a spreadsheet-like view.
View the documentation for `View()` by clicking **here.**

5. The `colnames()` function
   View the column names in the `brtri` dataframe with the `colnames()` function in the code chunk below.
   How many columns are in the dataframe?

`ncol(brtri)` shows number of columns. `colnames()` will display all column names with index numbers in square brackets at the start of each printed line, for example `[1]` and `[7]`. The number in brackets is the position of the first name on that line, which also allows you to count the number of columns.
View the documentation for `colnames()` by clicking **here.**
View the documentation for `ncol()` by clicking **here.**

```
colnames(brtri)
```

```
##   [1] "X"         "STATE"    "FMONTH"   "IDATE"    "IMONTH"   "IDAY"
##   [7] "IYEAR"     "DISPCODE" "SEQNO"    "PSU"      "CTELENM1" "PVTRESD1"
##  [13] "COLGHOUS"  "STATERE1" "CELPHON1" "LADULT1"  "NUMADULT" "RESPSLC1"
##  [19] "LANDSEX3"  "SAFETIME" "CTELNUM1" "CELLFON5" "CADULT1"  "CELLSEX3"
##  [25] "PVTRESD3"  "CCLGHOUS" "CSTATE1"  "LANDLINE" "HHADULT"  "SEXVAR"
##  [31] "GENHLTH"   "PHYSHLTH" "MENTHLTH" "POORHLTH" "PRIMINS2" "PERSDOC3"
```

```
##  [37] "MEDCOST1" "CHECKUP1" "EXERANY2" "LASTDEN4" "RMVTETH4" "CVDINFR4"
##  [43] "CVDCRHD4" "CVDSTRK3" "ASTHMA3"  "ASTHNOW"  "CHCSCNC1" "CHCOCNC1"
##  [49] "CHCCOPD3" "ADDEPEV3" "CHCKDNY2" "HAVARTH4" "DIABETE4" "DIABAGE4"
##  [55] "MARITAL"  "EDUCA"    "RENTHOM1" "NUMHHOL4" "NUMPHON4" "CPDEMO1C"
##  [61] "VETERAN3" "EMPLOY1"  "CHILDREN" "INCOME3"  "PREGNANT" "WEIGHT2"
##  [67] "HEIGHT3"  "DEAF"     "BLIND"    "DECIDE"   "DIFFWALK" "DIFFDRES"
##  [73] "DIFFALON" "HADMAM"   "HOWLONG"  "CERVSCRN" "CRVCLCNC" "CRVCLPAP"
##  [79] "CRVCLHPV" "HADHYST2" "HADSIGM4" "COLNSIGM" "COLNTES1" "SIGMTES1"
##  [85] "LASTSIG4" "COLNCNCR" "VIRCOLO1" "VCLNTES2" "SMALSTOL" "STOLTEST"
##  [91] "STOOLDN2" "BLDSTFIT" "SDNATES1" "SMOKE100" "SMOKDAY2" "USENOW3"
##  [97] "ECIGNOW3" "LCSFIRST" "LCSNUMCG" "LCSCTSC1" "LCSSCNCR" "LCSCTWHN"
## [103] "ALCDAY4"  "AVEDRNK4" "DRNK3GE5" "MAXDRNKS" "FLUSHOT7" "FLSHTMY3"
## [109] "IMFVPLA5" "PNEUVAC4" "HIVTST7"  "HIVTSTD3" "HIVRISK5" "PDIABTS1"
## [115] "PREDIAB2" "DIABTYPE" "INSULIN1" "CHKHEMO3" "EYEEXAM1" "DIABEYE1"
## [121] "DIABEDU1" "FEETSORE" "ARTHEXER" "SHINGLE2" "HPVADVC4" "HPVADSH1"
## [127] "TETANUS1" "CNCRDIFF" "CNCRAGE"  "CNCRTYP2" "CSRVTRT3" "CSRVDOC1"
## [133] "CSRVSUM"  "CSRVRTRN" "CSRVINST" "CSRVINSR" "CSRVDEIN" "CSRVCLIN"
## [139] "CSRVPAIN" "CSRVCTL2" "PSATEST1" "PSATIME1" "PCPSARS2" "PSASUGS1"
## [145] "PCSTALK2" "CIMEMLO1" "CDWORRY"  "CDDISCU1" "CDHOUS1"  "CDSOCIA1"
## [151] "CAREGIV1" "CRGVREL5" "CRGVPRB4" "CRGVALZD" "CRGVNURS" "CRGVPER2"
## [157] "CRGVHOU2" "CRGVHRS2" "CRGVLNG2" "ACEDEPRS" "ACEDRINK" "ACEDRUGS"
## [163] "ACEPRISN" "ACEDIVRC" "ACEPUNCH" "ACEHURT1" "ACESWEAR" "ACETOUCH"
## [169] "ACETTHEM" "ACEHVSEX" "ACEADSAF" "ACEADNED" "LSATISFY" "EMTSUPRT"
## [175] "SDLONELY" "SDHEMPLY" "FOODSTMP" "SDHFOOD1" "SDHBILLS" "SDHUTILS"
## [181] "SDHTRNSP" "HOWSAFE1" "MARIJAN1" "MARJSMOK" "MARJEAT"  "MARJVAPE"
## [187] "MARJDAB"  "MARJOTHR" "USEMRJN4" "LASTSMK2" "STOPSMK2" "MENTCIGS"
## [193] "MENTECIG" "HEATTBCO" "SSBSUGR2" "SSBFRUT3" "FIREARM5" "GUNLOAD"
## [199] "LOADULK2" "RCSGEND1" "RCSXBRTH" "RCSRLTN2" "CASTHDX2" "CASTHNO2"
## [205] "SOMALE"   "SOFEMALE" "HADSEX"   "PFPPRVN4" "TYPCNTR9" "NOBCUSE8"
## [211] "QSTVER"   "QSTLANG"  "HPVDSHT"  "ICFQSTVR" "METSTAT"  "URBSTAT"
## [217] "MSCODE"   "STSTR"    "STRWT"    "RAWRAKE"  "WT2RAKE"  "IMPRACE"
## [223] "CHISPNC"  "CRACE1"   "CAGEG"    "CLLCPWT"  "DUALUSE"  "DUALCOR"
## [229] "LLCPWT2"  "LLCPWT"   "RFHLTH"   "PHYS14D"  "MENT14D"  "HLTHPL2"
## [235] "HCVU654"  "TOTINDA"  "EXTETH3"  "ALTETH3"  "DENVST3"  "MICHD"
## [241] "LTASTH1"  "CASTHM1"  "ASTHMS1"  "DRDXAR2"  "MRACE1"   "HISPANC"
## [247] "RACE"     "RACEG21"  "RACEGR3"  "RACEPRV"  "SEX"      "AGEG5YR"
## [253] "AGE65YR"  "AGE80"    "AGEG"     "HTIN4"    "HTM4"     "WTKG3"
## [259] "BMI5"     "BMI5CAT"  "RFBMI5"   "CHLDCNT"  "EDUCAG"   "INCOMG1"
## [265] "RFMAM23"  "MAM402Y"  "CRVSCRN"  "RFPAP37"  "HPV5YR1"  "PAPHPV1"
## [271] "HADCOLN"  "CLNSCP2"  "HADSIGM"  "SGMSCP2"  "SGMS102"  "RFBLDS6"
## [277] "STOLDN2"  "VIRCOL2"  "SBONTI2"  "CRCREC3"  "SMOKER3"  "RFSMOK3"
## [283] "CURECI3"  "LCSLAST"  "LCSNUMC"  "LCSAGE"   "LCSYSMK"  "PACKDAY"
## [289] "PACKYRS"  "LCSYQTS"  "LCSSMKG"  "LCSELIG"  "LCSCTSN"  "LCSPSTF"
## [295] "DRNKANY6" "DROCDY4"  "RFBING6"  "DRNKWK3"  "RFDRHV9"  "FLSHOT7"
## [301] "PNEUMO3"  "AIDTST4"
```

5a. The `head()` function and the `$` operator

To view the first few rows of a dataframe, use the `head()` command; below, we look at the first 20 rows of the `STATE` column in the `brtri` dataframe.

The `$` operator picks one column out of a dataframe by name: `brtri$STATE` means “the `STATE` column

of `brtri`."
The second argument, `20`, is how many values to show. Without it, `head()` shows 6.
View the documentation for `head()` by clicking **here.**
View the documentation for `$` by clicking **here.**

```
head(brtri$STATE, 20)
```

```
##  [1] 9 9 9 9 9 9 9 9 9 9 9 9 9 9 9 9 9 9 9 9
```

5b. A frequency table with `table()`
To view a frequency table of a particular column within a dataframe, use the `table()` command; below, we look at a frequency table of the `STATE` column in the `brtri` dataframe.
The output has two lines: the top line lists each distinct value, and the line beneath it is how many rows have that value.
Add the option `useNA="ifany"` to display a count of missing values, if there are any; missing data will be listed as "NA" or "<NA>".
View the documentation for `table()` by clicking **here.**

```
table(brtri$STATE, useNA="ifany")
```

```
##
##     9    34    36
##  7213  8666 43913
```

6. Using `sapply()` to view column type
   View the class of each column in the `brtri` dataframe by implementing the `sapply()` function with the `class()` function in the code chunk below.
   Every column in R has a **class**, or type. The most common are `numeric`/`integer` (numbers), `character` (text), `logical` (TRUE/FALSE), and `factor` (categories; see Step 16).
   `class()` on its own checks one object. `sapply()` repeats it for every column and returns the results as a list of column names and classes.
   Notice that `class` is written without parentheses inside `sapply()`: we are handing R the function itself, and `sapply()` calls it for us.
   View the documentation for `sapply()` by clicking **here.**
   View the documentation for `class()` by clicking **here.**

```
sapply(brtri, class)
```

```
##         X     STATE    FMONTH     IDATE    IMONTH      IDAY     IYEAR  DISPCODE
## "integer" "integer" "integer" "integer" "integer" "integer" "integer" "integer"
##     SEQNO       PSU  CTELENM1  PVTRESD1  COLGHOUS  STATERE1  CELPHON1   LADULT1
## "integer" "integer" "integer" "integer" "integer" "integer" "integer" "integer"
##  NUMADULT  RESPSLC1  LANDSEX3  SAFETIME  CTELNUM1  CELLFON5   CADULT1  CELLSEX3
## "integer" "integer" "integer" "integer" "integer" "integer" "integer" "integer"
##  PVTRESD3  CCLGHOUS   CSTATE1  LANDLINE   HHADULT    SEXVAR   GENHLTH  PHYSHLTH
## "integer" "integer" "integer" "integer" "integer" "integer" "integer" "integer"
##  MENTHLTH  POORHLTH  PRIMINS2  PERSDOC3  MEDCOST1  CHECKUP1  EXERANY2  LASTDEN4
## "integer" "integer" "integer" "integer" "integer" "integer" "integer" "integer"
```

```
##  RMVTETH4  CVDINFR4  CVDCRHD4  CVDSTRK3   ASTHMA3   ASTHNOW  CHCSCNC1  CHCOCNC1
## "integer" "integer" "integer" "integer" "integer" "integer" "integer" "integer"
##  CHCCOPD3  ADDEPEV3  CHCKDNY2  HAVARTH4  DIABETE4  DIABAGE4   MARITAL     EDUCA
## "integer" "integer" "integer" "integer" "integer" "integer" "integer" "integer"
##  RENTHOM1  NUMHHOL4  NUMPHON4  CPDEMO1C  VETERAN3   EMPLOY1  CHILDREN   INCOME3
## "integer" "integer" "integer" "integer" "integer" "integer" "integer" "integer"
##  PREGNANT   WEIGHT2   HEIGHT3      DEAF     BLIND    DECIDE  DIFFWALK  DIFFDRES
## "integer" "integer" "integer" "integer" "integer" "integer" "integer" "integer"
##  DIFFALON    HADMAM   HOWLONG  CERVSCRN  CRVCLCNC  CRVCLPAP  CRVCLHPV  HADHYST2
## "integer" "integer" "integer" "integer" "integer" "integer" "integer" "integer"
##  HADSIGM4  COLNSIGM  COLNTES1  SIGMTES1  LASTSIG4  COLNCNCR  VIRCOLO1  VCLNTES2
## "integer" "integer" "integer" "integer" "integer" "integer" "integer" "integer"
##  SMALSTOL  STOLTEST  STOOLDN2  BLDSTFIT  SDNATES1  SMOKE100  SMOKDAY2   USENOW3
## "integer" "integer" "integer" "integer" "integer" "integer" "integer" "integer"
##  ECIGNOW3  LCSFIRST  LCSNUMCG  LCSCTSC1  LCSSCNCR  LCSCTWHN   ALCDAY4  AVEDRNK4
## "integer" "integer" "integer" "integer" "integer" "integer" "integer" "integer"
##  DRNK3GE5  MAXDRNKS  FLUSHOT7  FLSHTMY3  IMFVPLA5  PNEUVAC4   HIVTST7  HIVTSTD3
## "integer" "integer" "integer" "integer" "integer" "integer" "integer" "integer"
##  HIVRISK5  PDIABTS1  PREDIAB2  DIABTYPE  INSULIN1  CHKHEMO3  EYEEXAM1  DIABEYE1
## "integer" "integer" "integer" "integer" "integer" "integer" "integer" "integer"
##  DIABEDU1  FEETSORE  ARTHEXER  SHINGLE2  HPVADVC4  HPVADSH1  TETANUS1  CNCRDIFF
## "integer" "integer" "integer" "logical" "logical" "logical" "logical" "integer"
##   CNCRAGE  CNCRTYP2  CSRVTRT3  CSRVDOC1   CSRVSUM  CSRVRTRN  CSRVINST  CSRVINSR
## "integer" "integer" "integer" "integer" "integer" "integer" "integer" "integer"
##  CSRVDEIN  CSRVCLIN  CSRVPAIN  CSRVCTL2  PSATEST1  PSATIME1  PCPSARS2  PSASUGS1
## "integer" "integer" "integer" "integer" "integer" "integer" "integer" "integer"
##  PCSTALK2  CIMEMLO1   CDWORRY  CDDISCU1   CDHOUS1  CDSOCIA1  CAREGIV1  CRGVREL5
## "integer" "logical" "logical" "logical" "logical" "logical" "integer" "integer"
##  CRGVPRB4  CRGVALZD  CRGVNURS  CRGVPER2  CRGVHOU2  CRGVHRS2  CRGVLNG2  ACEDEPRS
## "integer" "integer" "integer" "integer" "integer" "integer" "integer" "logical"
##  ACEDRINK  ACEDRUGS  ACEPRISN  ACEDIVRC  ACEPUNCH  ACEHURT1  ACESWEAR  ACETOUCH
## "logical" "logical" "logical" "logical" "logical" "logical" "logical" "logical"
##  ACETTHEM  ACEHVSEX  ACEADSAF  ACEADNED  LSATISFY  EMTSUPRT  SDLONELY  SDHEMPLY
## "logical" "logical" "logical" "logical" "integer" "integer" "integer" "integer"
##  FOODSTMP  SDHFOOD1  SDHBILLS  SDHUTILS  SDHTRNSP  HOWSAFE1  MARIJAN1  MARJSMOK
## "integer" "integer" "integer" "integer" "integer" "integer" "integer" "integer"
##   MARJEAT  MARJVAPE   MARJDAB  MARJOTHR  USEMRJN4  LASTSMK2  STOPSMK2  MENTCIGS
## "integer" "integer" "integer" "integer" "integer" "integer" "integer" "integer"
##  MENTECIG  HEATTBCO  SSBSUGR2  SSBFRUT3  FIREARM5   GUNLOAD  LOADULK2  RCSGEND1
## "integer" "integer" "integer" "integer" "integer" "integer" "integer" "logical"
##  RCSXBRTH  RCSRLTN2  CASTHDX2  CASTHNO2    SOMALE  SOFEMALE    HADSEX  PFPPRVN4
## "logical" "integer" "integer" "integer" "integer" "integer" "integer" "integer"
##  TYPCNTR9  NOBCUSE8    QSTVER   QSTLANG   HPVDSHT  ICFQSTVR   METSTAT   URBSTAT
## "integer" "integer" "integer" "integer" "logical" "integer" "integer" "integer"
##    MSCODE     STSTR     STRWT   RAWRAKE   WT2RAKE   IMPRACE   CHISPNC    CRACE1
## "integer" "integer" "numeric" "numeric" "numeric" "integer" "integer" "integer"
##     CAGEG   CLLCPWT   DUALUSE   DUALCOR   LLCPWT2    LLCPWT    RFHLTH   PHYS14D
## "integer" "numeric" "integer" "numeric" "numeric" "numeric" "integer" "integer"
##   MENT14D   HLTHPL2   HCVU654   TOTINDA   EXTETH3   ALTETH3   DENVST3     MICHD
## "integer" "integer" "integer" "integer" "integer" "integer" "integer" "integer"
```

```
##   LTASTH1   CASTHM1   ASTHMS1   DRDXAR2    MRACE1   HISPANC      RACE   RACEG21
## "integer" "integer" "integer" "integer" "integer" "integer" "integer" "integer"
##   RACEGR3   RACEPRV       SEX   AGEG5YR   AGE65YR     AGE80      AGEG     HTIN4
## "integer" "integer" "integer" "integer" "integer" "integer" "integer" "integer"
##      HTM4     WTKG3      BMI5   BMI5CAT    RFBMI5   CHLDCNT    EDUCAG   INCOMG1
## "integer" "integer" "integer" "integer" "integer" "integer" "integer" "integer"
##   RFMAM23   MAM402Y   CRVSCRN   RFPAP37   HPV5YR1   PAPHPV1   HADCOLN   CLNSCP2
## "integer" "integer" "integer" "integer" "integer" "integer" "integer" "integer"
##   HADSIGM   SGMSCP2   SGMS102   RFBLDS6   STOLDN2   VIRCOL2   SBONTI2   CRCREC3
## "integer" "integer" "integer" "integer" "integer" "integer" "integer" "integer"
##   SMOKER3   RFSMOK3   CURECI3   LCSLAST   LCSNUMC    LCSAGE   LCSYSMK   PACKDAY
## "integer" "integer" "integer" "integer" "integer" "integer" "integer" "integer"
##   PACKYRS   LCSYQTS   LCSSMKG   LCSELIG   LCSCTSN   LCSPSTF  DRNKANY6   DROCDY4
## "integer" "integer" "integer" "integer" "integer" "integer" "integer" "integer"
##   RFBING6   DRNKWK3   RFDRHV9   FLSHOT7   PNEUMO3   AIDTST4
## "integer" "integer" "integer" "integer" "integer" "integer"
```

`sapply()` means “apply this function to every column.”
`str(brtri)` or `glimpse(brtri)` are alternatives that show the class and the first few values of each column.
View the documentation for `str()` by clicking **here.**
View the documentation for `glimpse()` by clicking **here.**
Every column contains–or is populated by–numbers; the BRFSS dataframe stores categorical variables as numbers that represent a text value or label.
Use the codebook to understand what the numbers represent.
For instance, the column SEXVAR contains values of NA, 1, and 2; NA represents missing data, 1 represents males, and 2 represents females.
Find the BRFSS codebooks and data files on the CDC website by clicking **here.**

6a. Why column class matters: `as.character()` and `as.numeric()`
This chunk shows what happens when a column has the wrong class. CTELENM1 is a telephone screening question (see the codebook) stored as numbers.
Line by line:
- `head()` and `table()` show what the column holds.
- `as.character()` converts the column to text, and `<-` saves the converted version back into the same column, replacing the original.
- `mean()` calculates an average. On a text column, R cannot do arithmetic, so it returns NA with a warning: “argument is not numeric or logical.” This is the lesson of the chunk.
- `class()` confirms the column is now `"character"`.
- `as.numeric()` converts the text back into numbers, and now `mean()` works again.

`na.rm=TRUE` tells `mean()` to skip missing values; we come back to this in Step 27.
A column that *looks* like numbers but is stored as text is a common problem with imported data. If `mean()` returns NA with that warning, check the class first.
View the documentation for `as.character()` by clicking **here.**
View the documentation for `as.numeric()` by clicking **here.**
View the documentation for `mean()` by clicking **here.**

```r
head(brtri$CTELENM1, 20)
```

```
##  [1] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
```

```r
table(brtri$CTELENM1)
```

```
##
##    1
## 7057
```

```r
brtri$CTELENM1 <- as.character(brtri$CTELENM1)

mean(brtri$CTELENM1, na.rm=TRUE)
```

```
## Warning in mean.default(brtri$CTELENM1, na.rm = TRUE): argument is not numeric
## or logical: returning NA
```

```
## [1] NA
```

```r
class(brtri$CTELENM1)
```

```
## [1] "character"
```

```r
brtri$CTELENM1 <- as.numeric(brtri$CTELENM1)

mean(brtri$CTELENM1, na.rm=TRUE)
```

```
## [1] 1
```

6b. A two-way table
Giving `table()` two columns, separated by a comma, makes a **cross-tabulation** (also called a two-way table or contingency table): states down the side as rows, sex across the top as columns, and the count of respondents in every combination.
This time we store the table in an object named `tbl` instead of printing it, so we can reuse it in the next step. Because the result is stored, nothing prints. Type `tbl` in the Console to see it.

```r
tbl <- table(brtri$STATE, brtri$SEXVAR, useNA="ifany")
```

6c. Turning counts into proportions with `prop.table()`
Counts are hard to compare when groups are different sizes. `prop.table()` takes a table of counts and divides each count by a total, giving **proportions** (numbers between 0 and 1; multiply by 100 for percentages).
The `margin` argument decides which total is used:
- `margin = 1`: divide by **row** totals, so each row adds up to 1. ("Within each state, what share of respondents are male or female?")
- `margin = 2`: divide by **column** totals, so each column adds up to 1. ("Within each sex, what share of respondents live in each state?")
- No `margin`: divide by the grand total, so the whole table adds up to 1.

Below, `margin = 2` gives column proportions: read down a column, and the values add up to 1.
View the documentation for `prop.table()` by clicking **here.**

```
prop.table(tbl, margin = 2)
```

```
##
##               1         2
##   9  0.1194527 0.1217332
##   34 0.1471283 0.1428986
##   36 0.7334190 0.7353681
```

To show percentages rounded to one decimal place, wrap it like this: `round(prop.table(tbl, margin = 2) * 100, 1)`. You'll see this pattern again in Steps 27 and 28.

7. The `table()` function
   Use the `table()` function to view the frequencies of the `STATE` column in the `brtri` dataframe in the code chunk below. Use the parameter `useNA` to view the number of rows with missing data.
   `brtri[,"STATE"]` is another way to pick a column: inside square brackets, the part before the comma chooses rows and the part after chooses columns. Leaving the row part empty means "all rows." `brtri[,"STATE"]` and `brtri$STATE` are equivalent.
   `STATE` holds FIPS codes, standard numeric codes for U.S. states (for example, 36 = New York).
   `useNA = "ifany"` is important; we'll return to missing data in Step 27.
   View the documentation for `table()` by clicking **here.**
   View the list of state FIPS codes by clicking **here.**

```
table(brtri[,"STATE"], useNA = "ifany")
```

```
##
##     9    34    36
##  7213  8666 43913
```

7a. Recoding with logical indexing
Codes like 9, 34, and 36 are hard to read. This chunk creates a new column, `STATENAME`, with state names.
Line by line:
- `brtri$STATENAME <- ""` creates the new column and fills every row with an empty text value `""`. If the column name doesn't exist yet, R creates it.
- `brtri$STATE==9` checks every row and returns `TRUE` where the state code is 9 and `FALSE` everywhere else. Note the **double** equals sign ==, which means "is equal to?"; a single = is used for arguments.
- `brtri$STATENAME[brtri$STATE==9] <- "Connecticut"` means "in the rows where the check is TRUE, put `"Connecticut"` in `STATENAME`." The square brackets select those rows. This is called **logical indexing**, and we will use it many times.

Then we check the result with `table()`, and confirm the new column is text with `class()`.
View the documentation for comparison operators such as == by clicking **here.**

```
brtri$STATENAME <- ""

brtri$STATENAME[brtri$STATE==9] <- "Connecticut"
brtri$STATENAME[brtri$STATE==34] <- "New Jersey"
brtri$STATENAME[brtri$STATE==36] <- "New York"

table(brtri$STATENAME, useNA="ifany")
```

```
##
## Connecticut  New Jersey    New York
##        7213        8666       43913
```

```
class(brtri$STATENAME)
```

```
## [1] "character"
```

7b. Recoding with `mutate()` and `case_when()`
This chunk does the same recode the tidyverse way, into a second column `STATENAME2`, so we can compare.
- `%>%` is the **pipe**. It can be read as "and then": take `brtri`, *and then* pass it to `mutate()`.
- `mutate()` creates a new column (or changes an existing one).
- `case_when()` works through a list of `condition ~ value` lines in order. Read `STATE==9 ~ "Connecticut"` as "if the state code is 9, the value is Connecticut."
- `.default = "Unknown"` is the value for any row that matched none of the conditions.
- `brtri <- ...` at the start saves the result. Without it, R would print the new dataframe but not keep the change.

Inside `mutate()`, you can write column names without `brtri$` because R already knows which dataframe you mean.
View the documentation for `mutate()` by clicking **here.**
View the documentation for `case_when()` by clicking **here.**
View the documentation for the pipe `%>%` by clicking **here.**

```
brtri <- brtri %>%
  mutate(STATENAME2 = case_when(
    STATE==9 ~ "Connecticut",
    STATE==34 ~ "New Jersey",
    STATE==36 ~ "New York",
    .default = "Unknown"
  ))
```

7c. Checking that two recodes agree
A cross-tabulation of the two new columns should have counts only where the names match (Connecticut with Connecticut, and so on). Any count off that diagonal would mean the two methods disagree.
If any state codes other than 9, 34, and 36 exist, they appear as `""` in `STATENAME` and `"Unknown"` in `STATENAME2`.

```
table(brtri$STATENAME, brtri$STATENAME2)
```

```
##
##               Connecticut New Jersey New York
##   Connecticut        7213          0        0
##   New Jersey            0       8666        0
##   New York              0          0    43913
```

7d. Checking a recode against the original codes
The most important check: cross-tabulate the new column against the column it came from. Each state name should line up with exactly one code. Checking a recode against the original is a habit worth building.

```
table(brtri$STATENAME, brtri$STATE)
```

```
##
##                    9    34    36
##   Connecticut  7213     0     0
##   New Jersey      0  8666     0
##   New York        0     0 43913
```

7e. Proportions of the whole table

Here `prop.table()` has no `margin`, so every cell is divided by the total number of respondents and the whole table adds up to 1 (compare with `margin = 2` in Step 6c). Each nonzero cell is that state's share of all respondents.

```
prop.table(table(brtri$STATE, brtri$STATENAME, useNA="ifany"))
```

```
##
##      Connecticut New Jersey  New York
##   9    0.1206349  0.0000000 0.0000000
##   34   0.0000000  0.1449358 0.0000000
##   36   0.0000000  0.0000000 0.7344294
```

7f. A yes/no (indicator) variable with `ifelse()`

An **indicator** (or "dummy") variable is coded 1 for yes and 0 for no. These are common in regression models.

`ifelse(test, yes, no)` checks a condition on every row and returns the `yes` value where it is TRUE and the `no` value where it is FALSE. Below: 1 if the respondent lives in New Jersey (FIPS code 34), 0 otherwise.

The first line, `<- NA`, isn't strictly needed because `ifelse()` fills every row, but creating a column as all-missing first is a safe habit.

The `table()` checks the indicator against `STATE`: the 1s should appear only in the row for 34.

View the documentation for `ifelse()` by clicking **here.**

```
brtri$JERSEYYN <- NA
brtri$JERSEYYN <- ifelse(brtri$STATE==34, 1, 0)

table(brtri$STATE, brtri$JERSEYYN, useNA="ifany")
```

```
##
##          0     1
##   9   7213     0
##   34     0  8666
##   36 43913     0
```

8. Viewing the number of rows with `nrow()`

   Use the `nrow()` function to view the number of rows in the `brtri` dataframe in the code chunk below.

   Each row is one survey respondent. `dim(brtri)` gives rows and columns together.

   View the documentation for `nrow()` by clicking **here.**

   View the documentation for `dim()` by clicking **here.**

```
nrow(brtri)
```

```
## [1] 59792
```

8a. Checking for duplicates with `duplicated()` and `any()`
`duplicated()` checks each value and returns `TRUE` if that value already appeared earlier in the column. `any()` then asks "is at least one of these TRUE?"
Here the answer is `TRUE`, and that is expected: thousands of respondents share the same state. `duplicated()` is most useful on an ID column, where every value should be unique. If `any(duplicated(id_column))` is `TRUE`, the same person may appear twice.
View the documentation for `duplicated()` by clicking **here.**
View the documentation for `any()` by clicking **here.**

```
any(duplicated(brtri$STATE))
```

```
## [1] TRUE
```

9. The BRFSS Codebook: Height
   The Behavioral Risk Factor Surveillance Survey (BRFSS) data in the `brtri` dataframe have a codebook with the following information:

Label: Reported Height in Feet and Inches
Section Name: Demographics

Type of Variable: Num
Variable Name: HEIGHT3

Question: About how tall are you without shoes? (If respondent answers in metrics, put a 9 in the first column)[Round fractions down.]

Value	Value Label	Frequency	Percentage	Weighted Percentage
200-711	Height (ft/inches)			
	Notes: 0 _ / _ _ = feet / inches	425,837	95.85	94.10
7777	Don't know/Not sure	4,198	0.94	1.49
9061-9998	Height (meters/centimeters)			
	Notes: The initial '9' indicates			
	this was a metric value.	7,793	1.75	2.91
9999	Refused	6,442	1.45	1.50
BLANK	Not asked or Missing	13,400		

Use the `table()` function to get a frequency table of the data in the `HEIGHT3` column in the code chunk below.
Given the information in the table above, what range of values would you say are in feet and inches in column `HEIGHT3`?
What range of values would you say are in centimeters?
View the documentation for `table()` by clicking **here.**

```
table(brtri$HEIGHT3, useNA = "ifany")
```

```
##
##  300  305  307  309  311  400  401  402  403  404  405  406  407  408  409  410
##    6    4    1    1    1   22    3    7    3    3   36   15   21   58  118  210
##  411  500  501  502  503  504  505  506  507  508  509  510  511  600  601  602
##  603 1931 1961 3659 3681 4611 4349 4841 4553 4103 3773 4189 3213 3497 1661 1410
##  603  604  605  606  607  608  609  610  611  700  701  702  704  705  706  708
##  861  495  180   79   41   27   11   12    9    5    2    2    4    3    1    1
##  711 7777 9090 9092 9095 9096 9098 9100 9102 9103 9104 9105 9106 9107 9108 9110
##    3  668    4    1    1    1    1    2    1    2    2   13    5    2    1    4
## 9111 9115 9118 9123 9125 9127 9129 9130 9132 9133 9135 9136 9137 9138 9140 9141
##    1    5    1    1    1    1    1    9    1    1    4    2    2    3   21    2
## 9142 9143 9144 9145 9146 9147 9148 9149 9150 9151 9152 9153 9154 9155 9156 9157
##    3    3    5   33    4    5   15   14  111   14   59   43   36   96   49   37
## 9158 9159 9160 9161 9162 9163 9164 9165 9166 9167 9168 9169 9170 9171 9172 9173
##   48   21  169   12   48   41   29  119   19   38   53   38  129   22   34   32
## 9174 9175 9176 9177 9178 9179 9180 9181 9182 9183 9184 9185 9186 9187 9188 9190
##   18   85   23   14   25    9   45    2   11   11    8   15    5    3    2    4
## 9191 9192 9193 9195 9196 9198 9200 9207 9251 9252 9999 <NA>
##    2    3    1    1    1    1    1    1    1    1  995 2081
```

9a. A mean that doesn't make sense
What is the average height of these respondents? The chunk below calculates the mean of HEIGHT3. The result is nonsense because 7777 and 9999 are codes for "don't know" and "refused," not heights, and values like 511 mean 5 feet 11 inches, not 511 of anything. Numbers in a dataset aren't always measurements. Keep this in mind for Step 27.

```
mean(brtri$HEIGHT3, na.rm=TRUE)
```

```
## [1] 1030.801
```

10. Extracting height: feet, inches
    According to the information in the BRFSS codebook, values from 200 to 711 in the HEIGHT3 column represent height in feet and inches.
    For example, the value '511' actually represents a height of 5 feet and 11 inches.
    The codebook also explains that values beginning with the number '9' in the HEIGHT3 column precede digits that represent height recorded in centimeters.
    The number '9' only signifies that the remaining digits are recorded in centimeters and should not be included in the measurement values.
    In order to compare height of each respondent, we will need to convert all height values to inches or centimeters.

We will accomplish this step by step:
1. Where height is recorded in feet and inches, for example '511', we'll separate values of feet and values of inches into separate HEIGHTft and HEIGHTin columns.
2. Where height is measured in centimeters, for example '9170', we will need to extract the last three digits–the values following the leading '9'–and store as values of centimeters in a separate HEIGHTcm

column.
3. After we have three new columns (HEIGHTft, HEIGHTin, HEIGHTcm), we can convert values conditionally by row until each respondent has height recorded in inches and centimeters.

Extract the 'foot' (the first digit) and 'inch' (the last two digits) components of each respondent's height from the HEIGHT3 column conditionally–only if brtri$HEIGHT3>=200 & brtri$HEIGHT3<=711.
Use conditional methods with logical indexing and the substr() function to create brtri$HEIGHTft and brtri$HEIGHTin columns and populate with the correct digits from the HEIGHT3 column.
When subsetting columns, NA values are not allowed, so remember to include !is.na(brtri$HEIGHT3) in the logical indexing.

The pieces of the long line below:
- >= means "greater than or equal to," and <= means "less than or equal to."
- & means "and": a row is TRUE only if *every* condition is TRUE.
- is.na() returns TRUE for missing values; the ! in front means "not," so !is.na() is TRUE for values that are *not* missing.
- as.character() turns 511 into the text "511" so we can pick out characters, substr(..., 1, 1) takes characters 1 through 1 (just the first one), and as.numeric() turns "5" back into the number 5.
- The same condition appears on both sides of <- so that the rows we take values *from* are the same rows we put values *into*.

Read the long line aloud from the inside out: "pick the rows where the condition is TRUE, take the first character, turn it into a number, and put it in HEIGHTft for those same rows." Rows that don't meet the condition get NA. Step 10a walks through each piece on six small values.

You can use Stack Overflow as a resource on conditional methods with logical indexing by clicking **here.**
View the documentation for substr() by clicking **here.**
View the documentation for logical operators (&, !) by clicking **here.**
View the documentation for is.na() by clicking **here.**

```
brtri$HEIGHTft[brtri$HEIGHT3>=200 & brtri$HEIGHT3<=711 & !is.na(brtri$HEIGHT3)] <-
  as.numeric(substr(as.character(brtri$HEIGHT3[brtri$HEIGHT3>=200 & brtri$HEIGHT3<=711 & !
```

Check the result: feet values should be between 2 and 7, with NA for everyone who didn't answer in feet and inches.

```
table(brtri$HEIGHTft, useNA="ifany")
```

```
##
##     3     4     5     6     7  <NA>
##    13  1099 44864  8283    21  5512
```

The inches come from the same rows, but this time substr(..., 2, 3) takes characters 2 through 3. For "511" that is "11"; for "602" it is "02", which as.numeric() turns into 2.

```
brtri$HEIGHTin[brtri$HEIGHT3>=200 & brtri$HEIGHT3<=711 & !is.na(brtri$HEIGHT3)] <- as.nume
```

```
table(brtri$HEIGHTin, useNA="ifany")
```

```
##
##    0    1    2    3    4    5    6    7    8    9   10   11 <NA>
## 5461 3627 5078 4545 5113 4572 4936 4616 4189 3903 4411 3829 5512
```

10a. Same logic on a small example
`c()` ("combine") makes a short **vector**, a list of values, so we can try each piece on six values instead of hundreds of thousands. The values are: two heights in feet/inches, one in centimeters, a "don't know" code, a "refused" code, and a missing value.
View the documentation for `c()` by clicking **here.**

```
x <- c(511, 602, 9170, 7777, 9999, NA)
```

The condition returns TRUE or FALSE for each value. Notice that the comparison on `NA` returns `NA`, because R can't say whether a missing value is between 200 and 711.

```
x >= 200 & x <= 711
```

```
## [1]  TRUE  TRUE FALSE FALSE FALSE    NA
```

Adding `!is.na(x)` turns that `NA` into `FALSE`, because `FALSE & anything` is `FALSE`. Now every position is a clear TRUE or FALSE.

```
x >= 200 & x <= 711 & !is.na(x)
```

```
## [1]  TRUE  TRUE FALSE FALSE FALSE FALSE
```

`substr()` with positions 1 to 1 gives the first character of each value. `NA` stays `NA`.

```
substr(as.character(x), 1, 1)
```

```
## [1] "5" "6" "9" "7" "9" NA
```

Positions 2 to 3 give the second and third characters.

```
substr(as.character(x), 2, 3)
```

```
## [1] "11" "02" "17" "77" "99" NA
```

10b. (Optional) Check the result
A height code like `260` falls inside 200-711 but would mean "2 feet 60 inches." Good practice is to check that every inch value is between 0 and 11.
`brtri$HEIGHTin > 11` returns TRUE for each inch value above 11. `sum()` counts the TRUEs, because R treats TRUE as 1 and FALSE as 0. `na.rm=TRUE` skips the `NA`s; without it, the answer would be `NA`. A result of 0 means no impossible inch values.
View the documentation for `sum()` by clicking **here.**

```
table(brtri$HEIGHTin, useNA = "ifany")
```

```
##
##    0    1    2    3    4    5    6    7    8    9   10   11 <NA>
## 5461 3627 5078 4545 5113 4572 4936 4616 4189 3903 4411 3829 5512
```

```
sum(brtri$HEIGHTin > 11, na.rm=TRUE)
```

```
## [1] 0
```

11. Transforming height: total inches
    Create a new column called `HEIGHTtotin` and populate it with values in total inches using column operations and the new `HEIGHTft` and `HEIGHTin` columns we just created.
    You should be able to create the column and populate it with values in a single line of code.
    * means multiply and + means add. R does the calculation for every row at once, so there is no need to loop through rows. Parentheses work as they do in math class.
    If feet or inches are NA, the total is NA. Missing values carry through calculations.
    View the documentation for arithmetic operators by clicking **here.**

```
brtri$HEIGHTtotin <- (brtri$HEIGHTft*12) + brtri$HEIGHTin
```

The average height in inches now makes sense, unlike in Step 9a.

```
mean(brtri$HEIGHTtotin, na.rm=TRUE)
```

```
## [1] 66.88884
```

11a. Average height by sex
Logical indexing works inside any function. `brtri$HEIGHTtotin[brtri$SEXVAR==1]` means "the heights of respondents whose SEXVAR is 1 (male)," and `mean()` averages only those.

```
mean(brtri$HEIGHTtotin[brtri$SEXVAR==1], na.rm=TRUE)
```

```
## [1] 69.84941
```

```
mean(brtri$HEIGHTtotin[brtri$SEXVAR==2], na.rm=TRUE)
```

```
## [1] 64.09183
```

12. Extracting height: centimeters
    Create a new column called `HEIGHTcm` and populate it with NA values.
    Extract the height of each respondent recorded in centimeters from the `HEIGHT3` column conditionally–only if `brtri$HEIGHT3>=9061 & brtri$HEIGHT3<=9998`.
    Use conditional methods with logical indexing and the `substr()` function to populate with the correct digits from the `HEIGHT3` column.
    Remember that the last three digits of the `HEIGHT3` column represent respondent height in centimeters if the leading digit is a '9'.
    When subsetting columns, NA values are not allowed, so remember to include `!is.na(brtri$HEIGHT3)` in the logical indexing.
    This is the same pattern as Step 10 with a different range and different characters: `substr(..., 2, 4)` takes characters 2 through 4, skipping the leading 9. Predict what `substr("9170", 2, 4)` returns before you run it.

```
brtri$HEIGHTcm <- NA
brtri$HEIGHTcm[brtri$HEIGHT3>=9061 & brtri$HEIGHT3<=9998 & !is.na(brtri$HEIGHT3)] <- as.nu
```

13. Transforming height: total centimeters
    Create a new column called `HEIGHTtotcm` by simply copying the column `brtri$HEIGHTcm`.
    This new column `HEIGHTtotcm` will hold the height of each respondent in centimeters.
    For rows with height recorded in inches, we will convert from inches to centimeters and populate `HEIGHTtotcm` with converted values.
    Use the `round()` function to round to the nearest whole centimeter.
    View the documentation for `round()` by clicking **here.**

First, copy the centimeter column. People who answered in centimeters are now done; everyone else is NA for now.

```
brtri$HEIGHTtotcm <- brtri$HEIGHTcm
```

Next, fill the gaps. The condition `is.na(brtri$HEIGHTcm) & !is.na(brtri$HEIGHTtotin)` picks rows with *no* centimeter value but *with* an inch value. For those rows, multiply inches by 2.54 (there are 2.54 cm in an inch) and round to 0 decimal places, a whole number.

```
brtri$HEIGHTtotcm[is.na(brtri$HEIGHTcm) & !is.na(brtri$HEIGHTtotin)] <- round(brtri$HEIGHT
```

Check the result. `summary()` shows the minimum, quartiles, median, mean, maximum, and the number of NAs. Heights should fall roughly between 90 and 240 cm.
View the documentation for `summary()` by clicking **here.**

```
summary(brtri$HEIGHTtotcm)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.     NAs
##    90.0   163.0   170.0   169.6   178.0   252.0    3744
```

14. Transforming height: total meters
    Create a new column called `HEIGHTtotm` by simply converting the column `brtri$HEIGHTtotcm` to meters.
    Use the `round()` function to round values to four decimal places.
    There are 100 centimeters in a meter, so we divide (`/`) by 100. No condition is needed because every row with a centimeter value gets a meter value, and NA stays NA.

```
brtri$HEIGHTtotm <- round(brtri$HEIGHTtotcm/100, 4)
```

15. Transforming height: total centimeters to total inches
    Fill in `HEIGHTtotin` for respondents who answered in centimeters, by converting `brtri$HEIGHTtotcm` to inches.
    Use the `round()` function to round values to the nearest whole inch.
    The condition picks rows that *have* a centimeter value but *don't* yet have an inch value, and multiplies by 0.393701 (inches per centimeter). This fills empty cells in the existing `HEIGHTtotin` column rather than creating a new one, so the inch values from Step 11 are untouched.

```
brtri$HEIGHTtotin[!is.na(brtri$HEIGHTtotcm) & is.na(brtri$HEIGHTtotin)] <- round(brtri$HEI
```

Steps 13-15 “fill in the gaps”: people who answered in inches get a centimeter value, and people who answered in centimeters get an inch value. Rounding to whole centimeters in Step 13 is why our BMI will differ slightly from BRFSS’s in Step 25.

16. Respondent Sex: Creating a factor
    The Behavioral Risk Factor Surveillance Survey (BRFSS) data in the `brtri` dataframe have a code-book with the following information:

Label: Sex of Respondent
Section Name: Respondent Sex

Type of Variable: Num
Variable Name: SEXVAR

Question: Sex of Respondent

Value	Value Label	Frequency	Percentage	Weighted Percentage
1	Male: Code=1 if LANDSEX3=1 or CELLSEX3=1	217,487	47.52	49.12
2	Female: Code=2 if LANDSEX3=2 or CELLSEX3=2	240,183	52.48	50.88

Create a new variable `SEX` and populate it with empty values (“ “).
Use conditional logic and indexing to code the value of `SEX` for all rows where `SEXVAR==1` as ‘Male’.
Use conditional logic and indexing to code the value of `SEX` for all rows where `SEXVAR==2` as ‘Female’.
This is the same recoding pattern as Step 7a.

```
brtri$SEX <- ""
brtri$SEX[brtri$SEXVAR==1] <- "Male"
brtri$SEX[brtri$SEXVAR==2] <- "Female"
```

16a. Converting to a factor with `factor()`
Use the `factor()` function to convert `brtri$SEX` to a factor, stored in a new column `SEX2`.
A **factor** is R’s term for a categorical variable: a fixed set of categories called **levels**.
- `levels` lists the categories that exist in the data, in the order you want them to appear in tables and plots.
- `labels` gives each level a display name, in the same order. Here `"Male"` is displayed as `"M"` and `"Female"` as `"F"`.

`level` = works because R allows partial argument names, but `levels` = is the full spelling.
Any row that remained `""` becomes NA, because `""` isn’t one of the listed levels.
View the documentation for `factor()` by clicking **here.**
Read the factors chapter of *R for Data Science* by clicking **here.**

```
brtri$SEX2 <- factor(brtri$SEX,
                     level = c("Male","Female"),
                     labels = c("M","F"))
```

Check the new factor. `table(brtri$SEXVAR, brtri$SEX, useNA = "ifany")` would also show that the recode worked.

```
table(brtri$SEX2, useNA="ifany")
```

```
##
##     M     F
## 28798 30994
```

The same mean-by-group idea as Step 11a, this time selecting rows by the text label instead of the number code. The comments show the results from the 2024 data.

```
mean(brtri$HEIGHTtotin[brtri$SEX=="Male"], na.rm=TRUE)
```

```
## [1] 69.70789
```

```
mean(brtri$HEIGHTtotin[brtri$SEX=="Female"], na.rm=TRUE)
```

```
## [1] 64.00712
```

```
# [1] 69.84941
# [1] 64.09183
```

17. Plotting a histogram of (unweighted) height stratified by sex
    Use the code chunk below to plot a histogram of respondent height in meters stratified by sex using `ggplot` and `geom_histogram`.
    A **histogram** splits a continuous measure into ranges ("bins") and draws a bar for how many people fall in each bin. It shows the shape of the data: where most values are, and how spread out they are.
    `ggplot` builds a plot in layers added with +: data, then aesthetics, then a geometry, then styling.

- `nbins <- length(unique(brtri$HEIGHTtotm))` counts the distinct height values. `unique()` removes repeats and `length()` counts what is left. We use one bin per distinct height.
- `brtri %>% ggplot(...)`: take the data, *and then* start a plot.
- `aes(HEIGHTtotm, fill = SEX)`: **aesthetics** map columns to parts of the plot. Height goes on the x-axis, and SEX decides the fill color.
- `geom_histogram()`: draw a histogram. `alpha = 0.5` makes bars half see-through (0 is invisible, 1 is solid).
- `scale_fill_manual()`: choose the colors yourself. `na.value` is the color for missing SEX.
- `theme_classic()`: a clean, plain style with no grid lines.

- `theme(legend.position = "right")`: put the legend on the right.

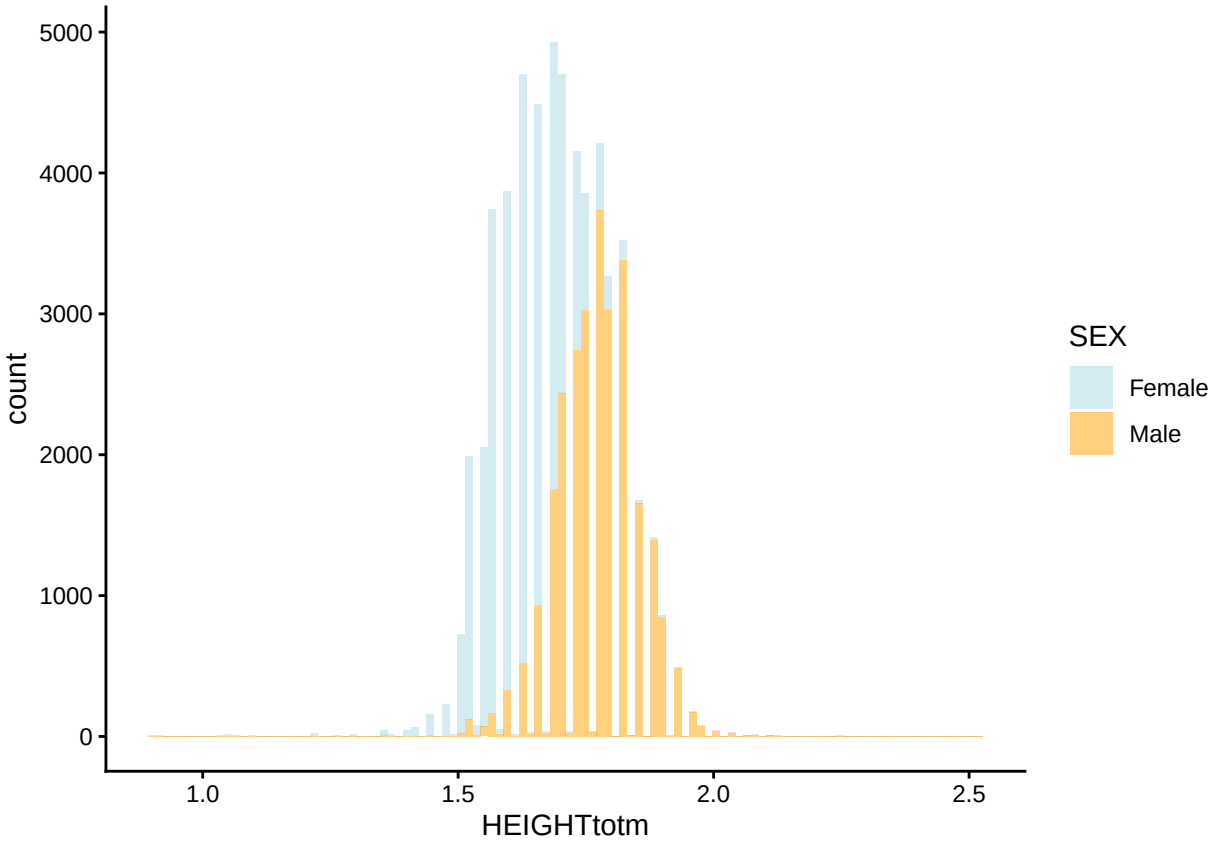
R will print a warning: "Removed rows containing non-finite values." These are respondents with missing height, and they disappear from the plot. This sets up Step 27.
By default `geom_histogram()` **stacks** the two groups, so the orange bars sit on top of the blue ones instead of overlapping. Step 17b shows the overlapping version with clearer labels.

View the documentation for `ggplot()` by clicking **here.**
View the documentation for `geom_histogram()` by clicking **here.**
View the documentation for `scale_fill_manual()` by clicking **here.**
View the built-in themes, including `theme_classic()`, by clicking **here.**
View a gallery of R code and histograms with several groups by clicking **here.**
Read the data visualization chapter of *R for Data Science* by clicking **here.**

```
nbins <- length(unique(brtri$HEIGHTtotm))

brtri %>%
  ggplot(aes(HEIGHTtotm, fill = SEX)) +
  geom_histogram(aes(fill = SEX),
                 bins = nbins,
                 alpha = 0.5) +
  scale_fill_manual(values = c('lightblue','orange'),
                    aesthetics = "fill",
                    breaks = waiver(),
                    na.value = "grey50") +
  theme_classic() +
  theme(legend.position = "right")
```

```
## Warning: Removed 3744 rows containing non-finite outside the scale range
## (`stat_bin()`).
```

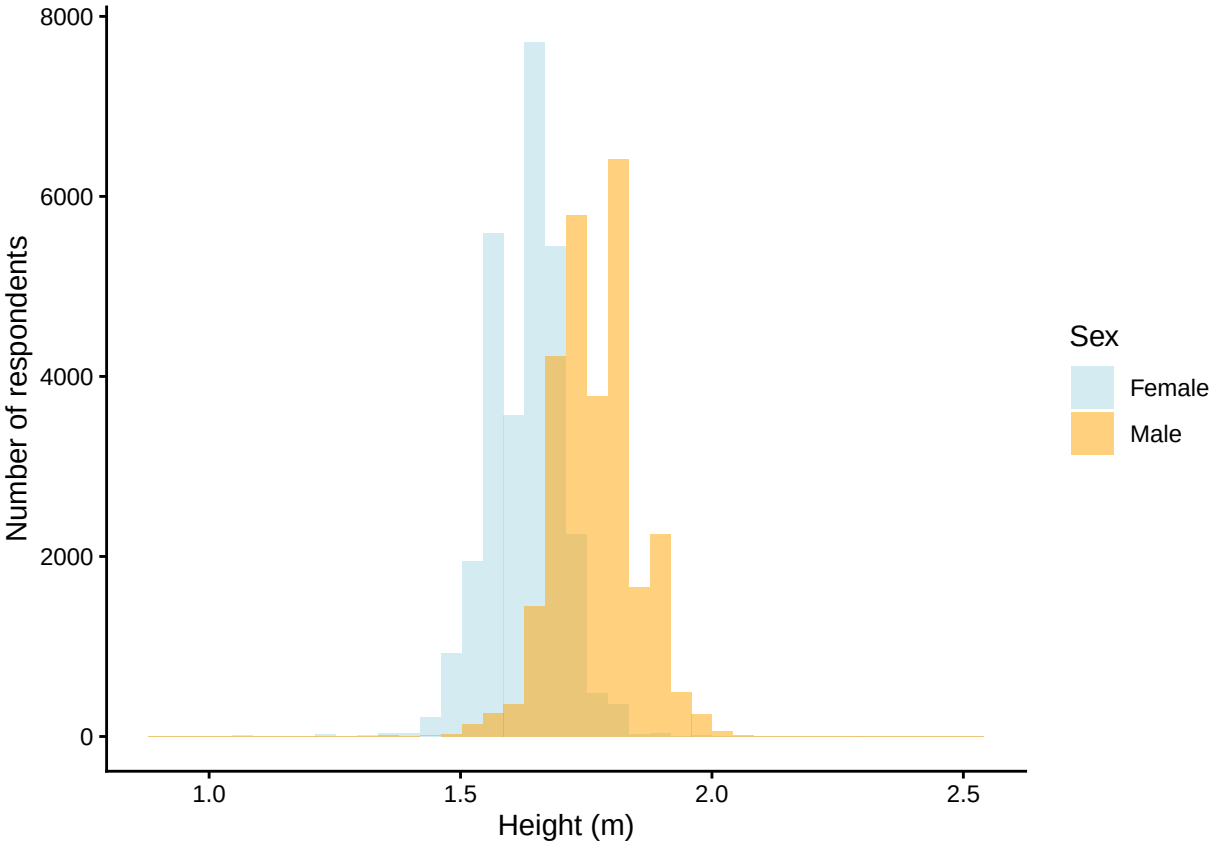


17b. (Optional) A cleaner version
Changes from Step 17:
- `filter(!is.na(SEX))` keeps only rows where SEX is not missing, before plotting. (Rows with `SEX == ""` are kept; check `table(brtri$SEX)` to see whether any exist.)
- `bins = 40` uses a fixed 40 bins, which usually gives a smoother picture than one bin per value.
- `position = "identity"` draws the two groups on top of each other (overlapping) instead of stacked, so the see-through colors let you compare the two shapes.
- `labs()` sets readable axis and legend titles.

View the documentation for `filter()` by clicking **here.**
View the documentation for `labs()` by clicking **here.**

```
brtri %>%
  filter(!is.na(SEX)) %>%
  ggplot(aes(HEIGHTtotm, fill = SEX)) +
  geom_histogram(bins = 40, alpha = 0.5, position = "identity") +
  scale_fill_manual(values = c('lightblue','orange')) +
  labs(x = "Height (m)", y = "Number of respondents", fill = "Sex") +
  theme_classic()
```

```
## Warning: Removed 3744 rows containing non-finite outside the scale range
## (`stat_bin()`).
```



18. Compare our calculated `HEIGHTtotin` column values to BRFSS height column values in inches
    Use the `head()` function to view the first rows of the `HEIGHTtotin` and `HTIN4` columns in the `brtri` dataframe.
    `brtri[,c("HEIGHTtotin","HTIN4")]` selects all rows (nothing before the comma) and two columns (the names inside `c()` after the comma). `head()` then shows the first 6 rows, so the two columns sit side by side.
    View the documentation for `head()` by clicking **here.**

```
head(brtri[,c("HEIGHTtotin","HTIN4")])
```

```
##   HEIGHTtotin HTIN4
## 1          64    64
## 2          NA    NA
## 3          67    67
## 4          67    67
## 5          64    64
## 6          60    60
```

19. Compare our calculated `HEIGHTtotm` column values to BRFSS height column values in meters
    Use the `head()` function to view the first rows of the `HEIGHTtotm` and `HTM4` columns in the `brtri` dataframe.
    `HTM4` looks 100 times too large. The BRFSS codebook stores `HTM4` with 2 implied decimal places, so 178 means 1.78 m. The same is true for `WTKG3` (Step 23) and `BMI5` (Step 25). Always read the codebook before comparing values.

```r
head(brtri[,c("HEIGHTtotm","HTM4")])
```

```
##   HEIGHTtotm HTM4
## 1       1.63  163
## 2         NA   NA
## 3       1.70  170
## 4       1.70  170
## 5       1.63  163
## 6       1.52  152
```